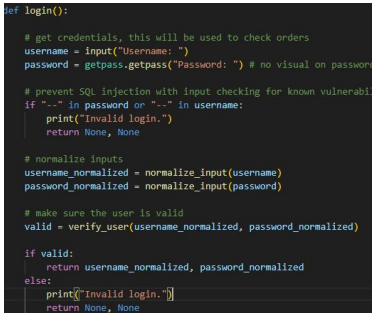
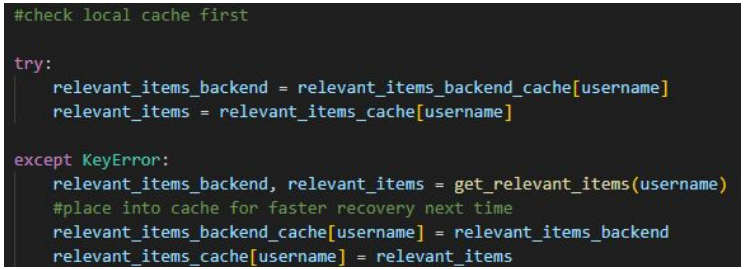
Design choices - Security, data retrieval, caching

- **Secure Login:** I use Python getpass which hides the password, I apply a basic normalization to the raw inputs, as well as identifying and preventing basic SQL-injection based attacks before granting access to the user.
- **Data retrieval:** I used hardcoded dictionaries in place of a real database connection, and this was primarily to keep the design simple and easily reproducible.
- **Caching:** I keep a cache of recently polled "recent purchases", with the idea that a user might ask for several items that might be lost, saving me time from a (theoretical) database query. This also allows me to have a faster "re-entry time" for when a user doesn't respond in a understandable way when prompted if their item is in the list of retrieved items.





Design choices - User Experience, Human-readability

```python
def get_relevant_items(username):

    # get order list, which has the product ids,
    order_list = RECENT_ORDERS[username][0]
    # make presentable as well
    items_human_readable = []
    for id in order_list:
        items_human_readable.append(ITEMS[id].lower())

    # do not show product id to users, separate
    return order_list, items_human_readable
```

- **Human-Database mapping:** I map product IDs, which, in the real world, tend to not be human-readable by keeping two lists, one with the IDs, and one with the names corresponding.
  - This allows the user to type that their item is “Brand Laptop” and not “A9801BBAC”
  - **Complexity:** This could also be done in a dictionary, which would have been faster with complexity O(1) rather than O(N) with the list, but since N is so small in this case (less than 10 samples), I decided to not go more in depth on it. If I was scaling this system even further I'd add this feature to reduce latency.

```python
isIn = normalize_input(input("Is your lost package in this list? (Type 'quit' to quit) "))
# catch the case in which they dont listen to instructions and just give the name
if (isIn in relevant_items):
    index = relevant_items.index(isIn)
    handle_last_known_location(relevant_items_backend[index])
    return -1
```

Short circuiting: The user doesn’t need to say “yes” then “<itemname>” they can just say <itemname>

- **Short circuiting:** I allow for a user to, instead of answering "yes or no" to the prompt, to *also* have the ability to just type the name of the object they’re looking for if they see it. This was done with the UX in mind that they might not want to go through several hoops (*“Is it in this list? Yes, then which one is it?”*) but rather just type the product when they see it.

Design choices - User Experience, Personalization and Leniency

- **Username-specific loading:** User doesn't have to input the specific order ID in order to find their recent purchases, this is because those IDs are usually complicated and long.
    - Instead, showing the recently purchased values from that username and password (which we got from login) allows the finding of a lost item to be more frictionless on the user, which would improve UX.
- **User leniency:** I allow the user to be more flexible with their affirmations or denials, allowing "yes", "ye", "yess", "y","yep", "yeah" to all mean "yes".
    - These all stem from different ways I'd say yes in a hurry, and also allows the user to be a bit more comfortable when operating the software that I'm making.
    - Notably, small errors like another s at the end will end up in yes, same with another o at the end of no (noo). In those cases it's clear what the customer *meant* to say, and being stuck on the semantics *there* would add more unnecessary frustration and friction.

Challenges faced, future improvements

Challenge:

Keeping the complexity low and reproducibility was difficult. It was more a balancing act of keeping the project simple, but also having innovative features. Every added external feature is one more thing that could break, and I wanted an air-tight, reproducible setup.

Future improvements

Extending this project further would include: Proper databases with tear down instead of object dictionaries, more fault tolerance with on-site data sharding and replication, and “ARRIVED” section onto database relations to allow *specific* personalization. Being able to *know* what item the customer wants with minimal interaction would be the ultimate goal.

Many of these future improvements / challenges faced come from trying to keep the reproducibility simple, configuring wallets for OracleDBs or sample data for MySQL is painful for the interviewer and I wanted to avoid that.